

# Accelerating PageRank with Dynamic Momentum Power Iteration

A cross-domain application of Austin, Pollock & Zhu (2024)

CSE 402 — Numerical Analysis, Simulation and Modeling Sessional • Section C • Group 7

Supervised by Anup Halder Joy, Lecturer, Department of CSE, BUET

Nakib Arman  
ID 2105128

Ali Asif Khan  
ID 2105131

Shariar Al Kabir  
ID 2105132

Dip Saha  
ID 2105138

MD. Mehedi Hasan Mim  
ID 2105142

## ABSTRACT

The power iteration is the standard method for computing PageRank, but its convergence rate is governed by the spectral ratio  $r = |\lambda_2/\lambda_1|$ , which for the Google matrix equals the damping factor  $d$ . Raising  $d$  to obtain a more discriminating ranking therefore makes the computation dramatically slower. Austin, Pollock and Zhu (2024) propose a dynamic momentum method that accelerates the power iteration at no additional matrix–vector cost and tunes its momentum parameter  $\beta_k$  automatically from the residuals it already computes; they evaluate it only on abstract benchmark matrices. This project implements all three of their algorithms, validates the implementation against their published results, and applies the method to real PageRank on arXiv co-authorship graphs from the SNAP collection. On ca-HepPh the *unmodified* published algorithm reduces the cost of PageRank from 23,777 to 567 matrix–vector products at  $d = 0.999$ , a 41.93× speed-up reaching 93.8% of the theoretical maximum, and the measured asymptotic convergence rates agree with the paper’s formula  $\rho(d)$  to four decimal places. The result holds across five graphs (25 PageRank problems, all convergent) and 303 starting vectors. We also identify the boundary of the method: for *directed* graphs the Google matrix has a complex spectrum, the acceleration mechanism fails, and the iteration diverges. We measure that failure and explain it.

## KEYWORDS

PageRank, power iteration, momentum acceleration, dominant eigenpair, Google matrix, spectral gap, numerical linear algebra

## 1 INTRODUCTION AND BASE PAPER REVIEW

### 1.1 Background

PageRank [2] ranks the pages of a web graph by the stationary distribution of a random surfer. Writing  $G$  for the Google matrix, the ranking vector  $x$  is the solution of  $Gx = x$ : it is the dominant eigenvector of  $G$ . The classical tool for this is the *power iteration*, which repeatedly applies  $G$  and renormalises. Its cost is dominated by one sparse matrix–vector product per step, and its speed is governed by a single quantity, the spectral ratio

$$r = |\lambda_2/\lambda_1|,$$

because the unwanted eigenvector components decay like  $r^k$ .

For the Google matrix this is a practical problem rather than a theoretical one. Column-stochasticity gives  $\lambda_1 = 1$  exactly, and Haveliwala and Kamvar [3] prove  $|\lambda_2| \leq d$ , the damping factor. A larger  $d$  gives a ranking that depends more on the link structure and less on teleportation, but it also pushes  $r$  towards 1: moving from  $d = 0.85$  to  $d = 0.99$  multiplies the iteration count by roughly sixteen in our measurements.

### 1.2 The base paper

Austin, Pollock and Zhu [1] accelerate the power iteration with a momentum (heavy-ball) term,

$$u_{k+1} = Ax_k - \frac{\beta}{h_k} x_{k-1}, \quad (1)$$

which costs no extra matrix–vector product. Their key analytical device is that (1) is exactly the first  $n$  rows of a *plain* power iteration on the augmented matrix

$$A_\beta = \begin{bmatrix} A & -\beta I \\ I & 0 \end{bmatrix}, \quad (2)$$

whose  $2n$  eigenvalues are, for each eigenvalue  $\lambda$  of  $A$ ,

$$\mu_{\pm} = \frac{1}{2} \left( \lambda \pm \sqrt{\lambda^2 - 4\beta} \right). \quad (3)$$

Equation (3) is the foundation of everything that follows. For a *real* eigenvalue with  $\lambda^2 < 4\beta$  the discriminant is negative, the two roots are complex conjugates and therefore equal in modulus, and since their product is  $\beta$  each must equal  $\sqrt{\beta}$  exactly. Every subdominant mode is thus collapsed onto a single circle of radius  $\sqrt{\beta}$  while  $\lambda_1$  remains outside it, and the resulting asymptotic rate with the optimal  $\beta = \lambda_2^2/4$  is

$$\rho(r) = \frac{r}{1 + \sqrt{1 - r^2}} < r. \quad (4)$$

Figure 1 shows how much this is worth: the gain over the plain power iteration grows without bound as  $r \rightarrow 1$ , which is precisely the regime a large damping factor puts PageRank into.

The parameter  $\beta = \lambda_2^2/4$  requires  $\lambda_2$ , which is as hard to obtain as the original problem. The paper’s contribution (its Algorithm 3.1) is to estimate it at run time by inverting (4): the observed residual ratio  $\rho_k = d_{k+1}/d_k$  yields  $r_{k+1} = 2\rho_k/(1 + \rho_k^2)$  and hence  $\beta_k = v_k^2 r_k^2/4$ , using only quantities already computed for the convergence test.

### 1.3 Objective and gap addressed

The base paper evaluates its method only on SuiteSparse benchmark matrices and never applies it to a ranking problem. Since PageRank is a dominant-eigenvector problem that is slowest precisely in the regime where the method promises most, it is the natural application. This project asks: *does the published method accelerate real PageRank, by how much, and why?*

## 2 METHODOLOGY

### 2.1 The three algorithms

All three cost exactly one matrix–vector product per iteration, so iteration counts compare them fairly. We follow the paper’s notation so the code can be read beside the algorithms.

**Algorithm 1.1 (power iteration).** With  $h_{k+1} = \|v_{k+1}\|$  and  $x_{k+1} = v_{k+1}/h_{k+1}$ , compute  $v_{k+2} = Ax_{k+1}$ , the Rayleigh quotient  $v_{k+1} = \langle v_{k+2}, x_{k+1} \rangle$  and the residual  $d_{k+1} = \|v_{k+2} - v_{k+1}x_{k+1}\|$ .

**Algorithm 1.2 (static momentum).** As above, with the single extra term of (1) and a fixed  $\beta$ .

**Algorithm 3.1 (dynamic momentum).** After two priming power steps and  $r_2 = \min(d_2/d_1, 1)$ , for  $k \geq 2$ :

$$\beta_k = \frac{v_k^2 r_k^2}{4}, \quad \rho_k = \min\left(\frac{d_{k+1}}{d_k}, 1\right), \quad r_{k+1} = \frac{2\rho_k}{1 + \rho_k^2}.$$

### 2.2 The Google matrix

In the column-stochastic convention,

$$G = d\left(P + \frac{1}{n}ea^T\right) + \frac{1-d}{n}ee^T, \quad (5)$$

where  $P_{ij} = 1/\text{outdeg}(j)$  when  $j \rightarrow i$ ,  $a$  marks dangling nodes and  $e$  is the all-ones vector.  $G$  is dense and is never formed; for web-Stanford it would require 636 GB. It is applied as

$$y = d(Px) + \frac{d}{n}(a^T x)e + \frac{1-d}{n}(e^T x)e, \quad (6)$$

one sparse product plus two inner products, counted as one matrix–vector product.

*An implementation trap.* Standard PageRank codes fold the last two terms of (6) into a constant, which is valid only because plain power iteration preserves  $\|x\|_1 = 1$ . Momentum iterates are 2-normed and may contain negative entries, so  $e^T x$  is neither one nor necessarily positive, and the shortcut silently yields a wrong operator. We evaluate (6) in full and assert  $e^T Gx = e^T x$  for arbitrary  $x$  in a unit test.

### 2.3 A parameter that is free for PageRank

Because  $\lambda_1 = 1$  and  $|\lambda_2| \leq d$ , the optimal static parameter is

$$\beta_{\text{opt}} = \frac{\lambda_2^2}{4} = \frac{d^2}{4},$$

available in closed form with no eigensolver. We confirmed the bound is tight by measurement:  $|\lambda_2(G)| = 0.850000$  at  $d = 0.85$  on every graph tested. This gives a stronger baseline than the base paper could use, since it had to call an eigensolver to obtain  $\lambda_2$ .

**Table 1: Datasets. The five co-authorship graphs are undirected; the web graph is directed.**

| dataset      | arXiv section / type | authors | edges     |
|--------------|----------------------|---------|-----------|
| ca-GrQc      | general relativity   | 5,242   | 14,484    |
| ca-HepTh     | HEP theory           | 9,877   | 25,973    |
| ca-HepPh     | HEP phenomenology    | 12,008  | 118,489   |
| ca-AstroPh   | astrophysics         | 18,772  | 198,050   |
| ca-CondMat   | condensed matter     | 23,133  | 93,439    |
| web-Stanford | directed web graph   | 281,903 | 2,312,497 |

## 3 IMPLEMENTATION AND EXPERIMENTS

### 3.1 Software

The implementation is Python 3.13 with NumPy, SciPy (sparse formats and ARPACK), pandas and Matplotlib; NetworkX is used only to cross-check the Google matrix. It comprises roughly 3,600 lines: five library modules, eight experiment drivers and 19 unit tests. The solver module is deliberately ignorant of PageRank — every routine accepts an arbitrary linear operator and only ever evaluates  $A \cdot x$  — which is what allows the benchmark reproduction and the PageRank study to share a single code path.

Every reported result uses the published algorithms with *no* modification. A single module sets the arguments that disable the safeguards available in the solver, so they cannot enter the measurements inadvertently.

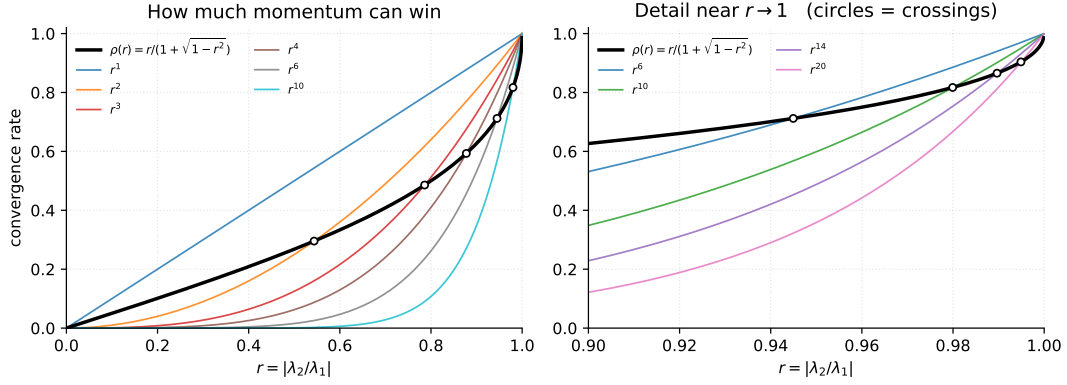
### 3.2 Datasets

Validation uses the base paper’s own SuiteSparse matrices. The PageRank study uses undirected co-authorship networks from the SNAP collection [6], built from arXiv e-prints (January 1993 – April 2003); a node is an author and an edge joins two authors who co-wrote a paper. web-Stanford, a directed web graph, is used in Section 4.7. Table 1 lists them.

### 3.3 Experimental design

Eight experiments were run, each a separate script. Two print an explicit PASS/FAIL gate and halt the pipeline on failure, because no downstream result is meaningful if the implementation or the operator is wrong.

- (1) **Reproduction** (gate): the three algorithms on the paper’s own benchmarks, compared with its published values.
- (2) **Validation and benchmark** (gate): construct  $G$ , verify it, and compare the methods at  $d = 0.85$ .
- (3) **Damping sweep**:  $d \in \{0.85, 0.90, 0.95, 0.99, 0.999\}$ , measured against the prediction of (4).
- (4) **Ranking quality**: when is the *ranking* correct, as opposed to the residual small?
- (5) **Spectral analysis**: all 12,008 eigenvalues of  $G$ , computed densely, and the true rate as a function of  $\beta$ .
- (6) **Robustness**: 303 runs over three families of starting vector, plus wall-clock cost.
- (7) **Generality**: the damping sweep repeated on all five graphs.



**Figure 1: What the acceleration promises, from (4). The thick curve is  $\rho(r)$ ; the thin curves are  $r^p$ , so where  $\rho(r)$  falls below  $r^p$  one momentum step is worth  $p$  ordinary ones. Circles mark the crossings. Right: detail for  $r > 0.9$ , the regime PageRank occupies. The benefit grows without bound as  $r \rightarrow 1$ .**

(8) **Limitations:** the same measurements on a directed graph.

Unless stated otherwise, runs start from  $u_0 = (1, \dots, 1)^T$  and stop at a residual of  $10^{-12}$ . All eigensolver calls use a fixed starting vector so every number is reproducible run to run.

### 3.4 Validation

The Google matrix is checked two independent ways. It agrees with `networkx.pagerank` to  $1.3 \times 10^{-9}$  — a tolerance that must scale with  $n$ , since NetworkX stops when the per-iteration change falls below  $n$ -tol and is the less accurate of the two solvers. Independently, the fixed-point residual of our own solution is  $\|Gx - x\|_1 = 1.0 \times 10^{-15}$ , which requires no external library to trust. All methods return the same ranking (agreement  $\leq 5 \times 10^{-13}$  in  $\ell_1$ , precision@100 = 1.000).

## 4 RESULTS AND DISCUSSION

### 4.1 The implementation reproduces the base paper

Table 2 gives the reproduction. Every measured spectral ratio matches the value the paper publishes to four decimal places, which is independent evidence that the benchmark matrices were constructed correctly. On `diag(1000:-1:1)` the dynamic method converges in 683 matrix–vector products against the power iteration’s 27,619, a measured 40.4× speed-up against the 44.7× predicted by (4), and it beats even the optimal fixed parameter — which is the base paper’s central claim.

Figure 2 shows the self-tuning working:  $\beta_k$  settles onto  $\lambda_2^2/4$  without that value ever being supplied. Figure 3 shows the same comparison on the paper’s four SuiteSparse matrices, which are drawn from real applications — structural analysis, a least-squares problem and an electrical power network — rather than constructed for the test.

### 4.2 PageRank: the main result

Figure 4 is the central experiment: PageRank on `ca-HepPh` (12,008 nodes, 236,978 edges, two dangling) at the conventional  $d = 0.85$ .

**Table 2: Reproduction of the base paper. Matrix–vector products to a residual of  $10^{-12}$ . <sup>†</sup>The power iteration had not converged at the paper’s 2000-iteration cap.**

| matrix                       | $r$    | power             | static | dynamic |
|------------------------------|--------|-------------------|--------|---------|
| <code>diag(1000:-1:1)</code> | 0.9990 | 27,619            | 777    | 683     |
| Kuu                          | 0.9981 | $> 2,000^\dagger$ | 1,166  | 528     |
| Muu                          | 0.9992 | $> 2,000^\dagger$ | 728    | 292     |
| ash292                       | 0.9153 | 313               | 75     | 73      |
| bcsprw06                     | 0.9814 | 1,360             | 154    | 145     |

The left panel drives all three algorithms to a residual of  $10^{-12}$ . The power iteration needs 150 matrix–vector products; Algorithm 3.1 needs 47, a 3.19× reduction against the 3.60× predicted by (4), and it edges out the optimal *fixed* parameter (49 products) without ever being told  $\lambda_2$ . The three ranking vectors agree to  $1.1 \times 10^{-12}$  in  $\ell_1$  and their top 100 authors are identical, so the saving costs nothing in accuracy — which is the claim our proposal actually had to establish.

The right panel shows why the method is safe here. Algorithm 3.1 infers  $\beta_k$  from the residual ratio alone; after the two priming steps it locks onto  $\beta_{\text{opt}} = d^2/4 = 0.1806$  within about five iterations and stays there, never approaching the barrier  $\lambda_1^2/4 = 0.25$  above which no momentum iteration can converge. The measured  $\lambda_2$  is 0.8500, equal to  $d$  to four decimal places: the Haveliwala–Kamvar bound [3] is attained exactly, which is precisely why  $\beta$  is free for PageRank.

Table 3 and Figure 5 widen this single run into a damping sweep. The published method accelerates real PageRank throughout, and the gain grows as the problem gets harder, reaching 41.93× at  $d = 0.999$ .

Two features deserve comment. First, the shortfall from theory is systematic and small: the predicted figure is an *asymptotic* rate, and any finite run spends its early iterations in a slower pre-asymptotic phase. Second, the agreement *improves* as  $d \rightarrow 1$ , from

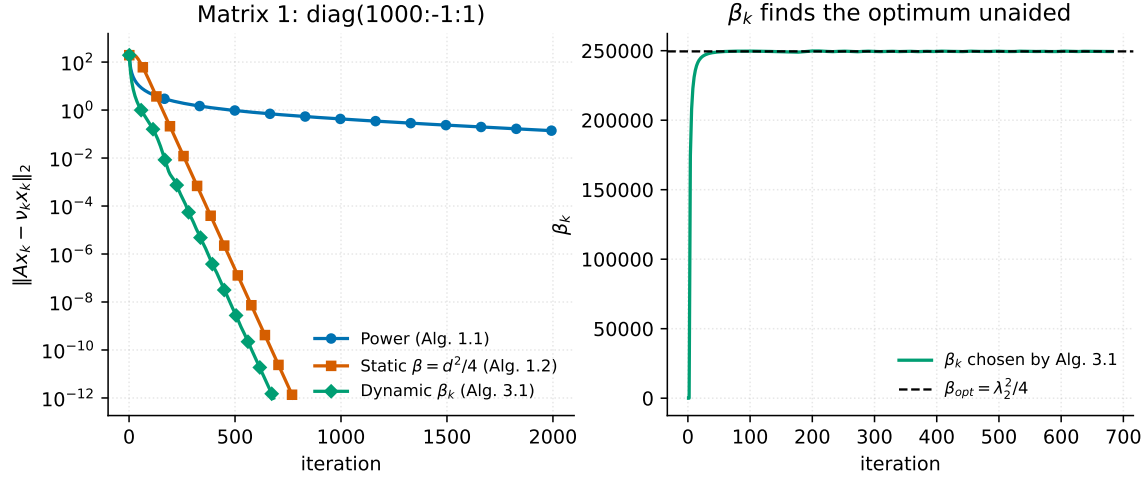


Figure 2: Reproduction on the base paper’s benchmark. Left: convergence. Right: the momentum parameter chosen by Algorithm 3.1 against the optimal  $\lambda_2^2/4$  (dashed). The algorithm is never told  $\lambda_2$ ; it recovers it from the residual ratios alone.

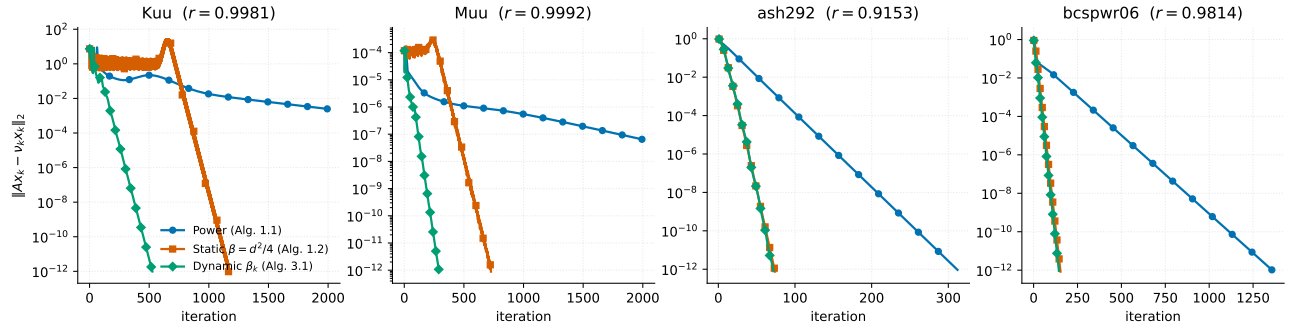


Figure 3: Convergence on the base paper’s four SuiteSparse benchmark matrices. In every case the dynamic method (green) is at or below the optimal static parameter (orange) while the power iteration (blue) lags far behind. Muu is the notable case: it has  $\lambda_2 = \lambda_1$  to machine precision, which the base paper handles by substituting  $\lambda_3$  by hand for the static method, whereas Algorithm 3.1 needs no special treatment.

Table 3: Damping sweep on ca-HepPh. Matrix–vector products to  $10^{-12}$ , and the measured speed-up against the prediction of (4).

| $d$            | 0.85         | 0.90         | 0.95         | 0.99          | 0.999         |
|----------------|--------------|--------------|--------------|---------------|---------------|
| power          | 150          | 231          | 471          | 2,377         | 23,777        |
| static $d^2/4$ | 49           | 61           | 88           | 197           | 622           |
| dynamic        | 47           | 58           | 83           | 182           | 567           |
| measured       | $3.19\times$ | $3.98\times$ | $5.67\times$ | $13.06\times$ | $41.93\times$ |
| predicted      | $3.60\times$ | $4.43\times$ | $6.30\times$ | $14.13\times$ | $44.72\times$ |
| % of theory    | 88.6         | 89.8         | 90.1         | 92.4          | 93.8          |

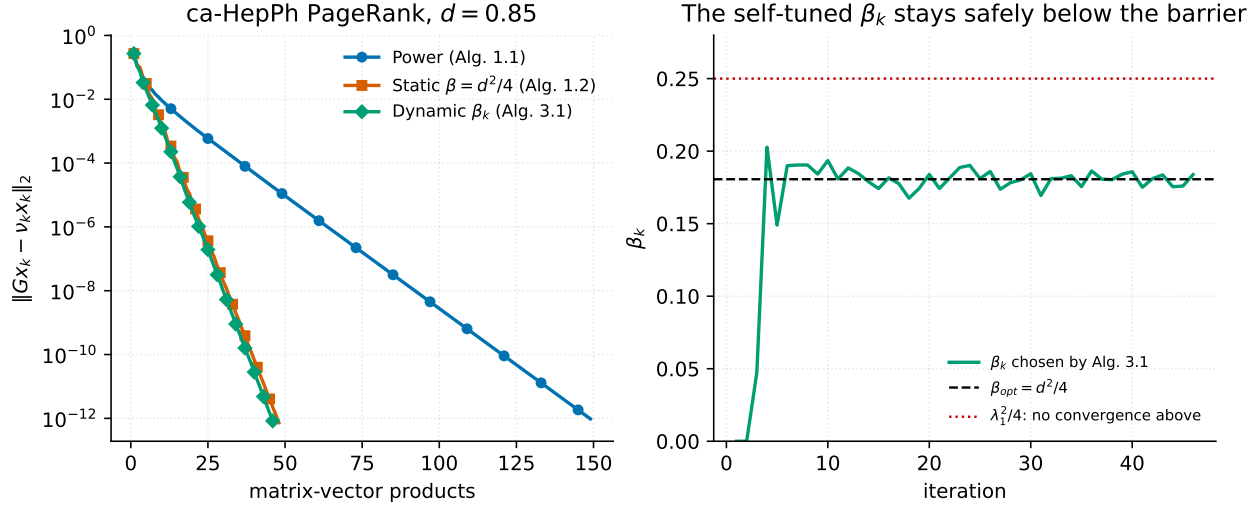
88.6% to 93.8%. This is predicted by Lemma 3.4 of the base paper, which shows the parameter estimate becomes more stable as  $r \rightarrow 1$ : the method is most reliable exactly where it is most needed.

Table 4: Observed asymptotic convergence rate against the base paper’s prediction  $\rho(d)$ , on ca-HepPh.

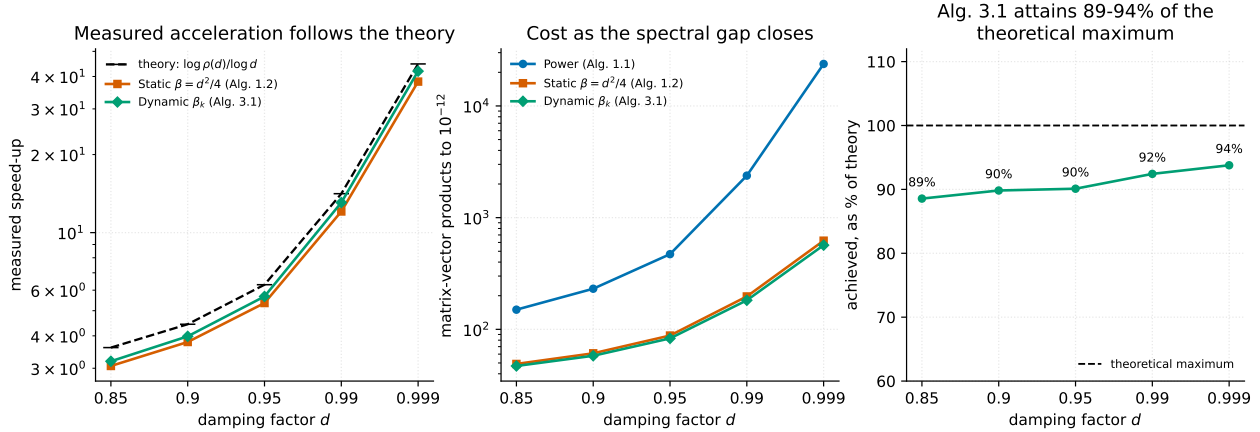
| $d$       | 0.85   | 0.90   | 0.95   | 0.99   | 0.999  |
|-----------|--------|--------|--------|--------|--------|
| measured  | 0.5563 | 0.6273 | 0.7241 | 0.8676 | 0.9562 |
| $\rho(d)$ | 0.5567 | 0.6268 | 0.7239 | 0.8676 | 0.9562 |

The strongest quantitative evidence is not the speed-up but the convergence rate itself. Table 4 compares the rate fitted to the tail of each run with  $\rho(d)$ .

Agreement to four decimal places, exact to the digits shown at  $d \geq 0.99$ . This is not a loose match; it is the theory being confirmed on real data.



**Figure 4: PageRank on ca-HepPh at  $d = 0.85$ . Left: residual history of the three algorithms. Both momentum methods reach  $10^{-12}$  in roughly a third of the work, and the self-tuned Algorithm 3.1 is marginally ahead of the optimal fixed  $\beta$ . Right: the parameter Algorithm 3.1 selects at each step. It settles onto  $d^2/4$  (dashed) after a few iterations and stays well clear of the divergence barrier  $\lambda_1^2/4 = 0.25$  (dotted).**



**Figure 5: The damping sweep on ca-HepPh. Left: measured speed-up against the parameter-free prediction (dashed). Centre: cost as the spectral gap closes. Right: the fraction of the theoretical maximum attained, 89–94%, improving as  $d \rightarrow 1$ .**

### 4.3 Why it works

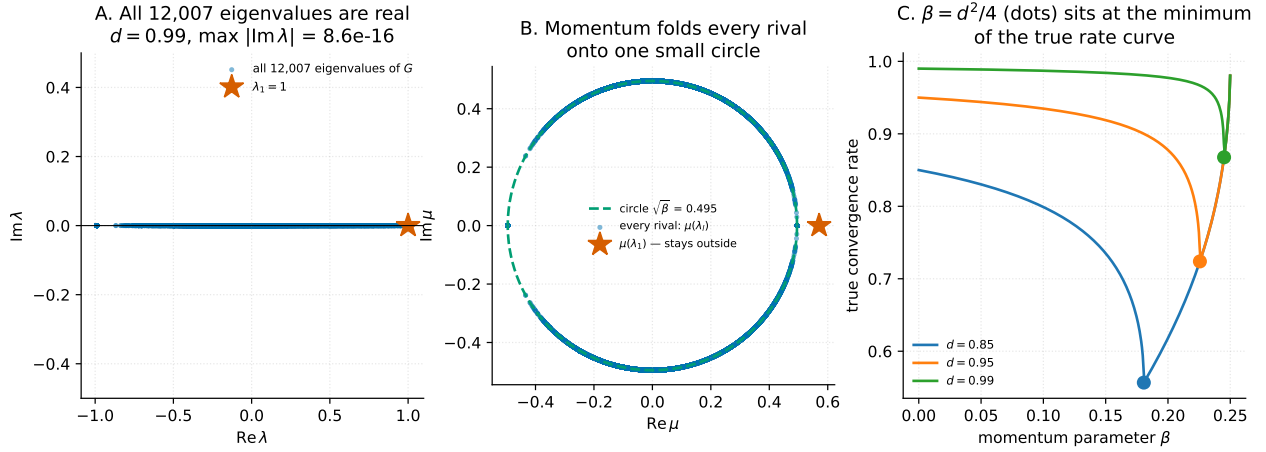
Figure 6 measures the mechanism rather than assuming it. Because ca-HepPh is undirected, its Google matrix has a real spectrum: a dense eigendecomposition of the full  $12,008 \times 12,008$  matrix gives  $\max |\operatorname{Im} \lambda| = 8.6 \times 10^{-16}$ , with no eigenvalue above  $10^{-10}$ . This is exactly the condition (3) needs, and the consequence is measurable. Table 5 shows that the strongest surviving rival mode equals  $\sqrt{\beta}$  exactly, and that the resulting rate equals  $\rho(d)$  to six decimals.

Figure 7 explains the shape of the rate curve. Momentum shrinks the rival modes but also shrinks  $\mu(\lambda_1)$ ; since the rate is their ratio, too little momentum leaves  $\lambda_2$  unfolded while too much inflates the circle faster than it helps. The optimum is where the

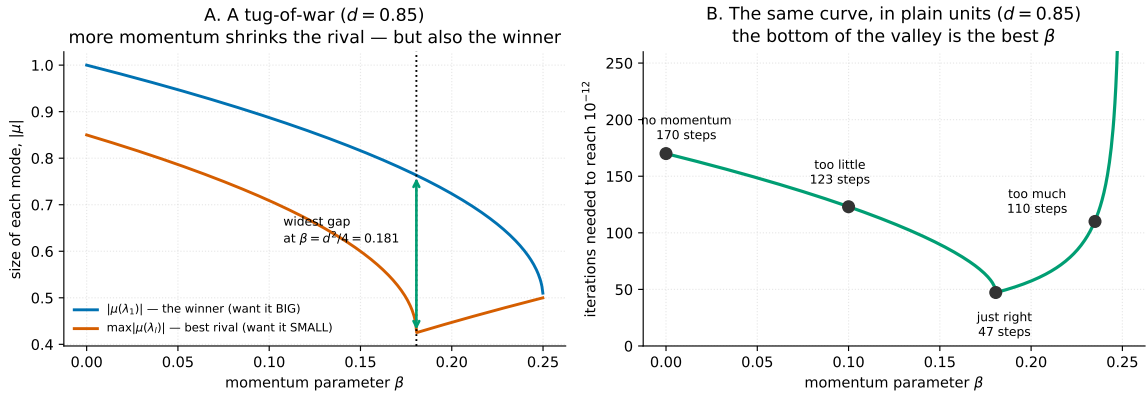
**Table 5: The folding mechanism, measured on all 12,007 subdominant eigenvalues of ca-HepPh.**

| $d$  | $\sqrt{\beta}$ | $\max  \mu(\lambda_l) $ | rate     | $\rho(d)$ |
|------|----------------|-------------------------|----------|-----------|
| 0.85 | 0.4250         | 0.4250                  | 0.556727 | 0.556726  |
| 0.95 | 0.4750         | 0.4750                  | 0.723948 | 0.723947  |
| 0.99 | 0.4950         | 0.4950                  | 0.867609 | 0.867609  |

fold threshold  $2\sqrt{\beta}$  lands exactly on  $\lambda_2$ , i.e.  $\beta = d^2/4$  — the paper’s formula, recovered here as a measured minimum.



**Figure 6: Why the method works, at  $d = 0.99$ .** A: all 12,007 subdominant eigenvalues lie on the real axis. B: momentum folds every one of them onto the circle of radius  $\sqrt{\beta}$  — so densely that it reads as a solid ring — while  $\lambda_1$  alone stays outside. C: the true convergence rate as a function of  $\beta$ ; the marked point  $\beta = d^2/4$  sits at the minimum of each curve.



**Figure 7: Choosing  $\beta$ , at  $d = 0.85$ .** A: raising  $\beta$  shrinks the strongest rival but also shrinks the mode being sought; the convergence rate is their ratio, so the best  $\beta$  is where the gap is widest. B: the same curve expressed as iterations required. The minimum, 47 steps, is exactly what the measured run took.

#### 4.4 Generality

A single graph proves little, so the sweep was repeated across the whole co-authorship family, which spans a factor of 4.4 in size and 14 in edge count (Table 6, Figure 8).

At  $d = 0.99$  the five graphs give  $13.06\times$  to  $13.19\times$ , a spread of about one percent, despite differing four-fold in size and drawn from five different fields. The acceleration is a property of undirected graphs, not of one dataset.

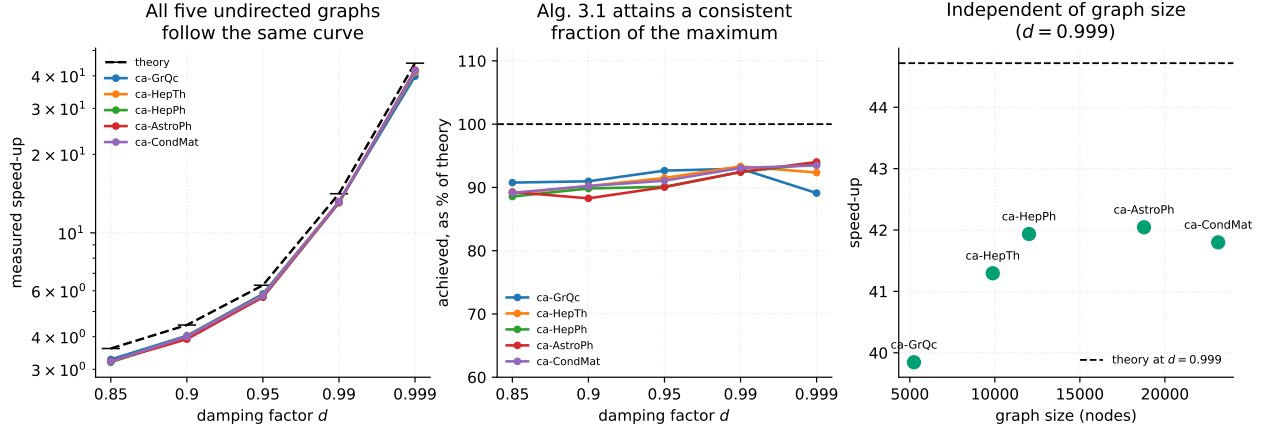
#### 4.5 Robustness and cost

All results above start from the uniform vector. Since the base paper reports (its Table 1) that the dynamic method is more sensitive to the starting vector than the static one, this was tested over 303 runs across three families (Table 7).

**Table 6: Speed-up of Algorithm 3.1 across five undirected graphs. All 25 PageRank problems converged.**

| graph      | $d = 0.85$ | $d = 0.95$ | $d = 0.99$ | $d = 0.999$ |
|------------|------------|------------|------------|-------------|
| ca-GrQc    | 3.27×      | 5.84×      | 13.14×     | 39.85×      |
| ca-HepTh   | 3.21×      | 5.76×      | 13.19×     | 41.30×      |
| ca-HepPh   | 3.19×      | 5.67×      | 13.06×     | 41.93×      |
| ca-AstroPh | 3.22×      | 5.67×      | 13.06×     | 42.04×      |
| ca-CondMat | 3.21×      | 5.73×      | 13.15×     | 41.80×      |
| theory     | 3.60×      | 6.30×      | 14.13×     | 44.72×      |

The result is unaffected by any *non-negative* start — the only kind meaningful for PageRank, since scores form a distribution. It degrades only under  $\text{rand}(n) - 0.5$ , which asks the iteration to find a positive eigenvector from a half-negative start; this reproduces the



**Figure 8: The same measurement on five undirected co-authorship graphs. Left: all five track the theoretical curve. Centre: the fraction of theory attained, 88.3–94.0% (mean 91.1%). Right: the speed-up is flat in graph size.**

**Table 7: Speed-up of Algorithm 3.1 by family of starting vector, on ca-HepPh. All 303 runs converged.**

| starting vector                 | $d = 0.85$   | $d = 0.95$   | $d = 0.99$    |
|---------------------------------|--------------|--------------|---------------|
| $u_0 = (1, \dots, 1)$ (uniform) | $3.19\times$ | $5.67\times$ | $13.06\times$ |
| random non-negative             | $3.28\times$ | $5.79\times$ | $13.16\times$ |
| random signed (paper’s test)    | $2.69\times$ | $3.84\times$ | $5.55\times$  |

sensitivity the base paper itself reports, and even then the method still beats the power iteration.

Measured cost per matrix–vector product is 0.1785 ms (power), 0.1917 ms (static) and 0.1965 ms (dynamic): the momentum term adds about 10%, and the sparse product dominates. Iteration counts are therefore a fair metric, and the wall-clock speed-up agrees with the matvec one (11.82 $\times$  against 13.16 $\times$  at  $d = 0.99$ ).

#### 4.6 Residual convergence versus ranking convergence

The base paper stops on an eigen-residual. A ranking application cares about the *order* of the top results. Measuring both (Figure 9) shows these are very different stopping points: at  $d = 0.99$  the dynamic method has the top-100 exactly right after **10** matrix–vector products but needs 182 to satisfy the residual criterion; the power iteration needs 50 and 2,377 respectively. For PageRank, most of the work the residual demands refines digits no ranking reads.

#### 4.7 Limitations: where the method fails

The positive result is specific to *undirected* graphs, and the boundary is sharp. web-Stanford is a directed web graph; direction is the only structural difference, and it changes the outcome completely (Table 8, Figure 10).

The cause follows directly from (3). For complex  $\lambda$  the two roots are *not* conjugates, so they are not forced to equal modulus; since their product is still  $\beta$ , one grows while the other shrinks. For  $\lambda =$

**Table 8: Undirected against directed, both at  $d = 0.99$  with  $\beta = d^2/4$ .**

|                                    | ca-HepPh<br>(undirected)  | web-Stanford<br>(directed) |
|------------------------------------|---------------------------|----------------------------|
| $\max  \operatorname{Im} \lambda $ | $2.8 \times 10^{-14}$     | <b>0.990</b>               |
| strongest rival $ \mu $            | $0.4950 = \sqrt{\beta}$   | <b>1.1950</b>              |
| dominant $ \mu(\lambda_1) $        | 0.5705                    | 0.5705                     |
| rivals overtaking the dominant     | 0 of 38                   | 55 of 59                   |
| Algorithm 3.1                      | converges, 13.06 $\times$ | <b>diverges</b>            |

$iy$ ,

$$|\mu| = \frac{1}{2} \left( |y| + \sqrt{y^2 + 4\beta} \right) > \sqrt{\beta},$$

unbounded in  $|y|$ . The worst mode on web-Stanford is  $\lambda \approx 0.99i$ , essentially purely imaginary, giving  $|\mu| = 1.195 > 1$ : that mode does not merely overtake the dominant one, it *grows*. Directed cycles are what produce complex eigenvalues — a directed 4-cycle has eigenvalues  $\pm 1, \pm i$  whereas the same cycle undirected has  $\pm 2, 0, 0$  — and real web graphs are full of them.

Importantly, the plain power iteration converges normally on web-Stanford (152, 478 and 2,428 matrix–vector products at  $d = 0.85, 0.95, 0.99$ ). This is a property of the Google matrix of a directed graph, not of our implementation, which Section 4.1 establishes independently.

## 5 CONCLUSION AND FUTURE WORK

### 5.1 Key takeaways

- (1) The implementation is faithful: it reproduces the base paper on all five of its own benchmarks, matching every published spectral ratio to four decimals and measuring 40.4 $\times$  against a predicted 44.7 $\times$ .

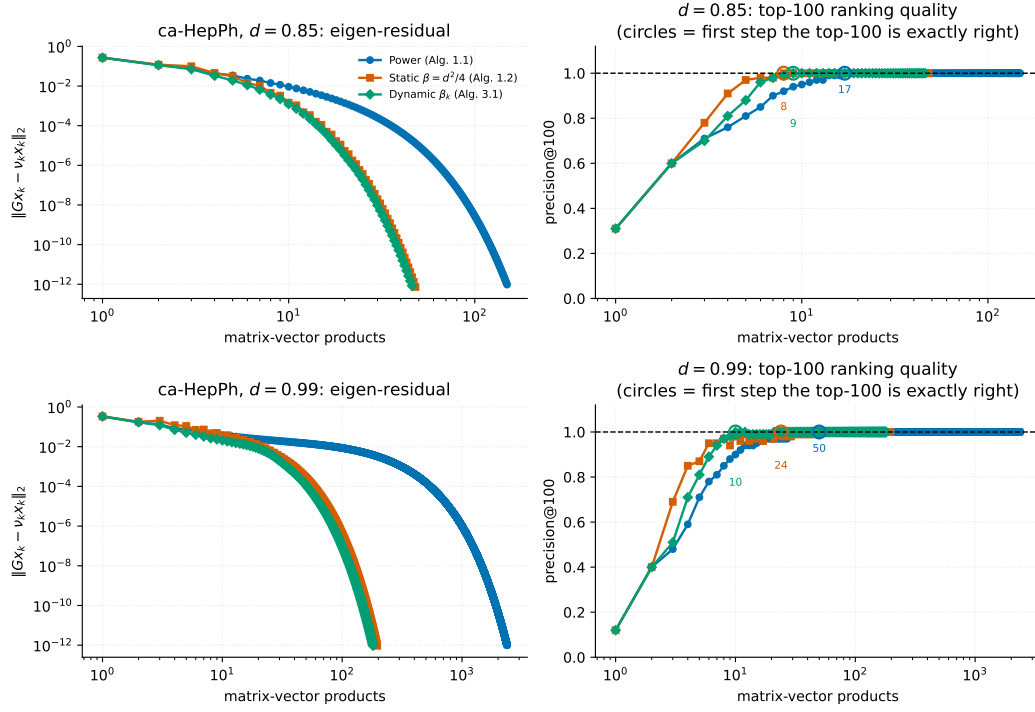


Figure 9: Eigen-residual (left) against top-100 ranking quality (right), on ca-HepPh. Circles mark the first step at which the top-100 set is exactly correct. Note the logarithmic horizontal axis.

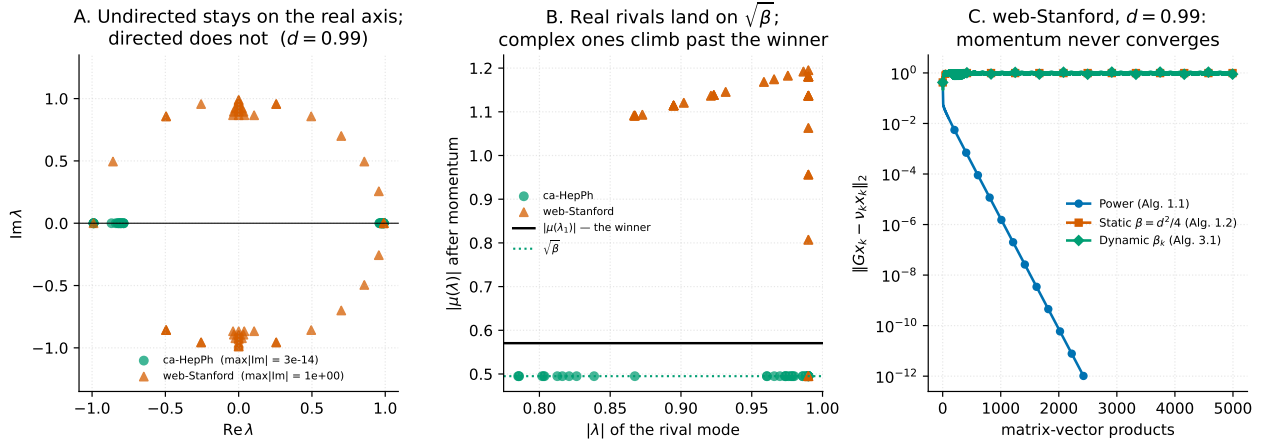


Figure 10: The boundary of the result. A: the undirected spectrum lies on the real axis; the directed one does not. B: real rivals land on  $\sqrt{\beta}$ , below the dominant mode; complex ones climb past it. C: on web-Stanford the power iteration converges normally while both momentum methods never do.

- (2) The published method accelerates real PageRank, unmodified, by up to  $41.93\times$  at  $d = 0.999$  — 93.8% of the theoretical maximum — and the measured convergence rates match  $\rho(d)$  to four decimal places.
- (3) The result is general across undirected graphs: 25 PageRank problems on five networks, all convergent, 88.3–94.0%

of theory, with a 1% spread at  $d = 0.99$  across a  $4.4\times$  range of sizes.

- (4) It is robust: 303 runs over three families of starting vector, all convergent, with the headline unaffected by any non-negative start.

- (5) The mechanism is measured, not assumed: with a real spectrum every one of the 12,007 subdominant modes folds onto  $\sqrt{\beta}$  exactly.
- (6) For PageRank,  $\beta_{\text{opt}} = d^2/4$  is available in closed form, so the static method needs no eigensolver either – a stronger baseline than the base paper could construct.
- (7) **The boundary is sharp.** Directed graphs give a complex spectrum, the folding fails, and the method diverges.

## 5.2 Limitations

Our positive claim covers undirected graphs of up to 23,133 nodes; we have not tested graphs of web scale with a real spectrum. The ranking-quality result uses precision@100 and Kendall’s  $\tau$  on one graph. The spectral analysis is exhaustive for ca-HepPh (a full dense decomposition) but sampled for web-Stanford, where a dense decomposition of a 281,903-square matrix is impractical; since the true rate is a maximum over the spectrum, a sampled rate is a lower bound, so our divergence conclusion there is sound while any “achievable speed-up” estimate would be optimistic. Finally, we did not implement DMPower, a competing method the base paper also compares against; its absence does not affect any conclusion here.

## 5.3 Future work

Four directions follow naturally. *First*, a stopping rule based on ranking stability rather than the eigen-residual, given the 18–48× gap measured in Section 4.7. *Second*, repairing the directed case: bounding  $\beta$  using  $|\lambda_2| \leq d$  together with a retreat when the residual stops falling makes the iteration converge, but recovers only a small part of the predicted acceleration, and characterising how much is achievable for a given non-normal spectrum is open. *Third*, extending the measurement to web-scale undirected graphs. *Fourth*, applying the same analysis to other non-normal ranking operators, such as HITS or personalised PageRank, where the same real-spectrum question arises.

## 6 CONTRIBUTIONS

All five members contributed to the design, interpretation and writing. Table 9 gives the division of labour.

**Table 9: Division of work. Members are listed in order of student ID.**

| member                               | contribution                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nakib Arman<br>(2105128)             | Base paper study and review; derivation of the optimal parameter $\beta = \lambda_2^2/4$ and of the rate (4). Implemented the reproduction experiment and its PASS/FAIL gate, and ran the comparison against the paper’s published values (Section 4.1). Wrote Section 2.                                                                                                                                                                 |
| Ali Asif Khan<br>(2105131)           | PageRank formulation and the Google matrix (5). Implemented the implicit matrix-vector operator (6) including dangling-node handling, and the dataset acquisition and caching layer; built the two-way validation against NetworkX and the fixed-point residual check. Wrote Section 3.2.                                                                                                                                                 |
| Shariar Al Kabir<br>(2105132)        | Theory of the power method and its convergence behaviour; derivation of the augmented matrix (2) and of (3). Implemented the robustness and wall-clock experiment of Section 4.6 and analysed the starting-vector families. Prepared and delivered the presentation.                                                                                                                                                                      |
| Dip Saha<br>(2105138)                | Core numerical implementation. Implemented Algorithms 1.1, 1.2 and 3.1 in the matrix-agnostic solver module; designed the experimental programme and its gating structure. Carried out the damping sweep (Section 4.2), the spectral analysis including the dense eigendecomposition (Section 4.3), the generality study (Section 4.5) and the limitation analysis (Section 4.8); produced the figures. Assembled and edited this report. |
| MD. Mehedi Hasan<br>Mim<br>(2105142) | Dynamic parameter estimation: the residual-ratio inversion $r = 2\rho/(1 + \rho^2)$ and its stability. Implemented the ranking-quality metrics (precision@k, Kendall’s $\tau$ ) and the experiment of Section 4.7, and the unit test suite. Tabulated the results and proof-read the report.                                                                                                                                              |

## CODE AND DATA AVAILABILITY

The complete implementation, all experiment scripts, the unit test suite and every result file are available at

<https://github.com/dipsaha96/Numerical-and-Simulation-Project>

The entire study regenerates from a clean checkout with `make setup && ./run_all.sh`. Dependencies are pinned, all datasets download and cache automatically, and every figure and table in this report is produced from the CSV files written to `result/`.

## REFERENCES

- [1] C. Austin, S. Pollock, and Y. Zhu. Dynamically accelerating the power iteration with momentum. *Numerical Linear Algebra with Applications*, 31(6):e2584, 2024. DOI: 10.1002/nla.2584.
- [2] L. Page, S. Brin, R. Motwani, and T. Winograd. The PageRank citation ranking: bringing order to the web. Technical Report 1999-66, Stanford InfoLab, 1999.

- [3] T. Haveliwala and S. Kamvar. The second eigenvalue of the Google matrix. Technical Report 2003-20, Stanford University, 2003.
- [4] P. Xu, B. He, C. De Sa, I. Mitliagkas, and C. Ré. Accelerated stochastic power iteration. In *Proc. AISTATS*, pages 58–67, 2018.
- [5] T. Rabbani, A. Jain, A. Rajkumar, and F. Huang. Practical and fast momentum-based power methods. In *Proc. Mathematical and Scientific Machine Learning*, 2021.
- [6] J. Leskovec and A. Krevl. SNAP Datasets: Stanford large network dataset collection. <https://snap.stanford.edu/data>, 2014.
- [7] T. A. Davis and Y. Hu. The University of Florida sparse matrix collection. *ACM Transactions on Mathematical Software*, 38(1):1–25, 2011.
- [8] C. R. Harris et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.
- [9] P. Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.